

Tunable Client-Centric Consistency in a MongoDB Replica Set

Zhao Yikun · Zhou Jiahao · Chen Zheke

25 September 2026

Abstract

This project investigates how MongoDB consistency controls affect four client-centric guarantees: read-your-writes (RYW), monotonic reads (MR), monotonic writes (MW), and writes-follow-reads (WFR). We deployed MongoDB 8.0.12 as a three-node Docker replica set and compared four configurations that vary read concern, write concern, read preference, and use of an explicit causally consistent session. Experiments covered normal operation, one- and two-node loss, Primary failover, and replication-network partition. Each trial used versioned documents and explicit dependency fields; timeout, rollback, and recovery were reported separately from consistency violations.

The strongest contrast was RYW under a stopped Secondary: C1 returned a stale version in 70 of 100 evaluable formal trials, whereas C4 returned none in 100. MR produced no regressions in 400 formal S2/S4 trials, while a targeted partition diagnostic produced five C1 regressions and five C4 timeouts. The fixed WFR matrix completed 600 formal attempts: 596 were evaluable, no confirmed or candidate violation was observed, four were inconclusive, 1,198 of 1,200 workload operations succeeded, and every trial recovered. MW experiments found no confirmed client write-order violation, but loss of a majority caused unavailable outcomes. These observations agree with the registered predictions within the tested protocols and show that durability, causal ordering, availability, and recovery require separate denominators.

Keywords MongoDB; replica sets; causal consistency; client-centric guarantees; fault injection

1. Project scope and research questions

The project asks how application-visible consistency changes when a replicated database varies acknowledgement, visibility, routing, and causal-session settings. MongoDB documents the four causal-consistency properties used in this report [1]. Our operational definitions are:

- **Read-your-writes (RYW):** after a client receives acknowledgement for a write, its later read should reflect that write or a newer value.
- **Monotonic reads (MR):** after a client observes version k , its later successful reads should not return a version below k .
- **Monotonic writes (MW):** writes issued sequentially by one logical client should be applied in that client order.
- **Writes-follow-reads (WFR):** a write issued after reading version k should be applied only to a history containing version k or a later version.

We ask three questions. First, can asynchronous Secondary replication expose stale state under weak settings? Second, do majority concerns and a causally consistent session replace stale success with waiting or explicit unavailability? Third, how do node loss, failover, and network partition change consistency and availability outcomes? The experiments provide bounded observations for specified protocols; they do not prove universal guarantees over every possible execution.

2. Database choice and deployment

2.1 Why MongoDB

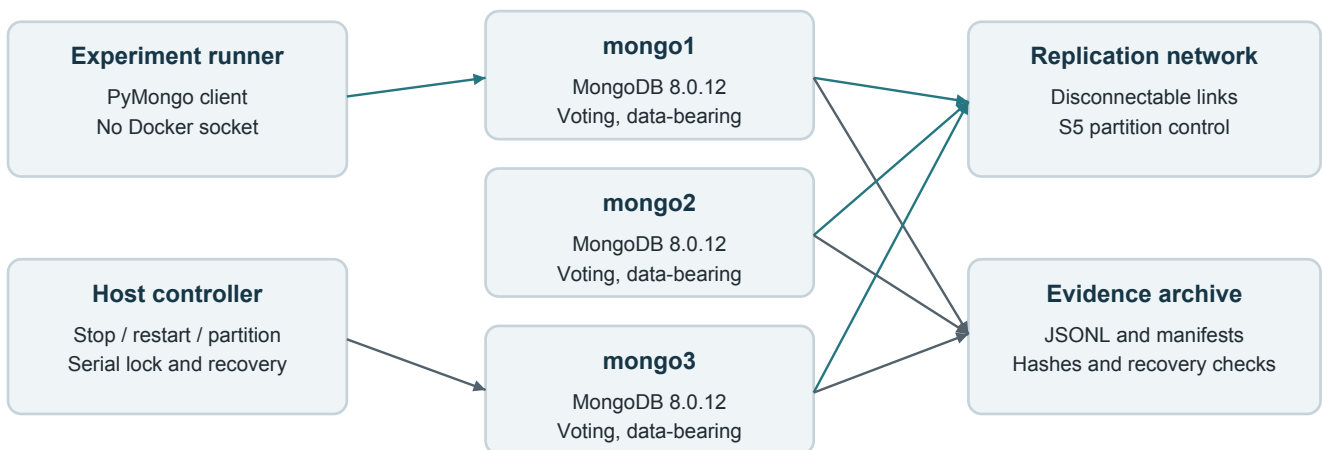
MongoDB was selected because a replica set provides practical replication while exposing per-operation read concern, write concern, read preference, and logical sessions through PyMongo [2-7]. These controls separate four mechanisms: where a read is routed, what state is visible, how durable a write acknowledgement is, and whether causal metadata is carried between operations. Replica sets also support elections, member shutdown, and asynchronous replication, which enable the required failure scenarios [6].

2.2 Architecture

The deployment contains three voting, data-bearing members in replica set `rs0`. Healthy operation has one Primary and two Secondaries. The normal cluster advertises `mongo1:27017`, `mongo2:27017`, and `mongo3:27017` inside its Docker network, with host mappings on `127.0.0.1` ports `27017-27019`. Clients run inside the Docker network so replica-set discovery uses the advertised addresses.

Fault experiments use an independent Compose project and separate volumes. A replication network is separated from per-node observation networks. The host controller can stop containers or disconnect replication links, while the experiment runner cannot access the Docker socket. A lock prevents concurrent fault experiments, recovery code reconnects networks and restarts nodes, and member roles are discovered dynamically.

Replica-set deployment and evidence flow



Healthy topology: one Primary and two Secondaries; roles discovered dynamically.

Figure 1. Deployment architecture and evidence flow. The same physical host runs all containers, which is a stated validity limitation.

Table 1. Recorded software and hardware environment

Component	Recorded environment
Database	MongoDB 8.0.12; three-member replica set rs0
Main host	Apple M4; 16 GiB RAM; macOS 15.7.3
Container platform	Docker Desktop 4.45.0; Engine 28.3.3; Compose 2.39.2
Docker VM allocation	10 CPUs; approximately 7.65 GiB RAM
Experiment runtime	Python 3.12.14; PyMongo 4.17.0
Supporting packages	dnspython 2.8.0; jsonschema 4.26.0
Analysis environment	Python 3.12; Matplotlib 3.11.1

Some trials were reproduced on another Apple Silicon host with a newer Docker release. Each archived manifest records the actual runtime state and source hash, so Table 1 describes the main environment rather than assigning one host version to every historical run.

2.3 Installation and initialization

The setup requires Git, Docker Compose v2, and Python 3.9 or later for the host controller. The experiment runtime is built into a pinned image. The startup script creates the containers, runs `rs.initiate` when required, and waits for one Primary and two Secondaries. Waiting only for ping is insufficient because a replica set may accept connections before electing its first Primary [6].

Before each fault batch the controller verifies topology. After every trial it requires restored membership and document convergence. The deployment and recovery procedure is documented in the repository [8].

3. Consistency configurations and predictions

3.1 Tested configurations

Table 2. Frozen C1-C4 configuration matrix

ID	read Concern	write Concern	read Preference	Causal session	Purpose
C1	local	w=1	secondaryPreferred	No	Weak baseline; exposes lag and permits rollback
C2	local	w=1	primary	No	Stable-Primary routing comparison
C3	majority	w=majority	secondaryPreferred	No	Majority durability without explicit dependency chain
C4	majority	w=majority	secondaryPreferred	Yes	Strong causal configuration

A local read can return a member's locally available state, which may later roll back [2]. A majority read returns majority-committed data, although a Secondary can lag the latest committed point [2,3]. w=1 acknowledges after one member accepts the write; w=majority waits for majority acknowledgement [5]. secondaryPreferred selects an eligible Secondary when available and can fall back to the Primary [4].

C3 and C4 deliberately hold concerns and read preference constant. Their intended difference is explicit propagation of causal dependency metadata. For C4, every related operation receives the same causally consistent ClientSession, and the session is used by one thread at a time [1,7]. Driver-created implicit sessions in C1-C3 are not treated as proof that application operations shared a causal chain.

3.2 Predictions registered before formal sampling

Table 3. Pre-registered guarantee and observation predictions

Config	RYW	MR	MW	WFR
C1	Not guaranteed	Not guaranteed	Expected for sequential modifying writes	Not guaranteed
C2	Conditional in stable single-Primary execution	Conditional	Expected for sequential modifying writes	Conditional
C3	Not guaranteed solely by majority concerns	Not guaranteed solely by majority concerns	Expected and majority durable	Not guaranteed solely by majority concerns
C4	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold

MongoDB documents all four durable causal guarantees for causally consistent sessions using majority reads and majority writes [1]. The C4 prediction therefore requires an acknowledged predecessor, majority read and write concerns, one serially used causal session, and explicit passage of that session to every related operation [1,7]. A timeout or write-concern failure is an availability outcome; it is neither a successful consistency check nor automatically a violation.

C1 was expected to expose stale RYW and MR observations when lag was deliberately created. MW violations were not expected for ordinary sequential modifying writes, although a w=1 acknowledgement can later roll back. C2 was expected to look strong during stable Primary routing without establishing a durable cross-failover claim. C3 improves durability and visibility but does not by itself propagate one client's dependencies.

4. Experimental method

4.1 Common evidence and scenarios

Each trial has a unique run and document identity. Operation records include the model, configuration, client and session identifiers, actual target node, written and returned versions, monotonic duration, status, error, violation decision, and current Primary. Manifests retain source hashes, source archives, runtime versions, effective arguments, and cluster snapshots. Pilot, formal, legacy, and diagnostic cohorts are labeled and are never pooled.

Table 4. Scenario definitions

Scenario	Fault action and purpose
S1 normal	Healthy one-Primary/two-Secondary replica set; baseline behavior
S2 Secondary stop	Stops one selected Secondary during the causal sequence; exposes routing and lag
S3 two-Secondary stop	Removes the majority; contrasts immediate operation with the settled state
S4 Primary crash	Stops the actual Primary and waits for election where required
S5 replication partition	Isolates replication links while preserving controlled observation paths

4.2 Model-specific protocols

RYW. Each trial initializes `x.version=0`, then one logical client writes `x.version=1` and waits for acknowledgement. The same client immediately reads `x`. A successful read below the acknowledged version is a violation. An unacknowledged, timed-out, or failed write cannot establish a valid RYW predecessor.

MR. Each trial places `x.version=1` on the required members. An independent producer writes version 2. One read client performs its first read, the controller injects S2 or S4, and the same client performs the second read. A lower version or disappearance of a previously observed document is a violation. C4 keeps the same session for both reads. The targeted S5 diagnostic forces a verified 2→1 opportunity.

MW. One logical client issues version 1/client_seq 1 and then version 2/client_seq 2 using synchronous sequential calls. Atomic `findOneAndUpdate` preimages, write history, and a final audit establish whether the predecessor existed before the second update. Rollback is reported separately from order inversion.

WFR. The fixed v2 protocol commits `x.version=1` and verifies convergence. The tested client reads `x` and records `k`, the controller injects S2/S4/S5, and the same client writes `y.depends_on_x=k`. C4 retains one session across the read and write. An atomic preimage and an independent observer corroborate the marker and `x` history. Missing corroboration or write timeout is inconclusive, not a pass or confirmed violation [12].

4.3 Metrics and evidence rules

- Violation rate = source-adjudicated violations / evaluable trials. A zero denominator is N/A, not 0%.
- Candidate, invalid, rollback, unavailable, and inconclusive outcomes remain separate classifications.
- Operation success is reported against attempted workload operations and planned slots in started trials.
- Successful-operation p95 excludes timeouts and failures and is descriptive rather than a clean benchmark.
- Recovery requires explicit restoration of one Primary, two Secondaries, and convergence evidence.
- Ordering uses same-process monotonic timestamps and matched control receipts; host and container wall clocks are not subtracted.

5. Results

The results below are tied to Git commit 634b9747130d6a3fc6fe71ca29a5c17596c0658b (origin/main on 25 September 2026) and the immutable unified snapshot results/summary/unified-0922-wfr-final/. Its catalog records 72 source cohorts and source SHA-256 values; summary.json normalizes 4,130 trials into 111 groups, of which 88 satisfy the selection rules for the core figures [11]. These heterogeneous data are not treated as one statistical denominator.

5.1 Normal-operation evidence

Table 5. Normal-operation evidence with sampling stage shown explicitly

Model/cohort	C1	C2	C3	C4	Stage and interpretation
RYW, 100/config	93/100	0/100	1/100	0/100	Legacy; stale-read outcomes, no reliable operation latency
MR, 100/config	0/100	0/100	0/100	0/100	Pilot baseline; no forced lag window
MW, 100/config	0/100	0/100	0/100	0/100	Formal baseline; sequential modifying writes
WFR continuity	0/1	Not run	Not run	0/1	Smoke only; excluded from formal normal evidence

The rows in Table 5 differ in schema and sampling stage and are not pooled. The RYW C1 result is consistent with immediate Secondary reads exposing lag. The single C3 stale observation indicates that majority visibility alone does not carry an explicit application dependency. Zero normal-operation MR/MW violations do not establish universal guarantees where nodes were not forced to disagree.

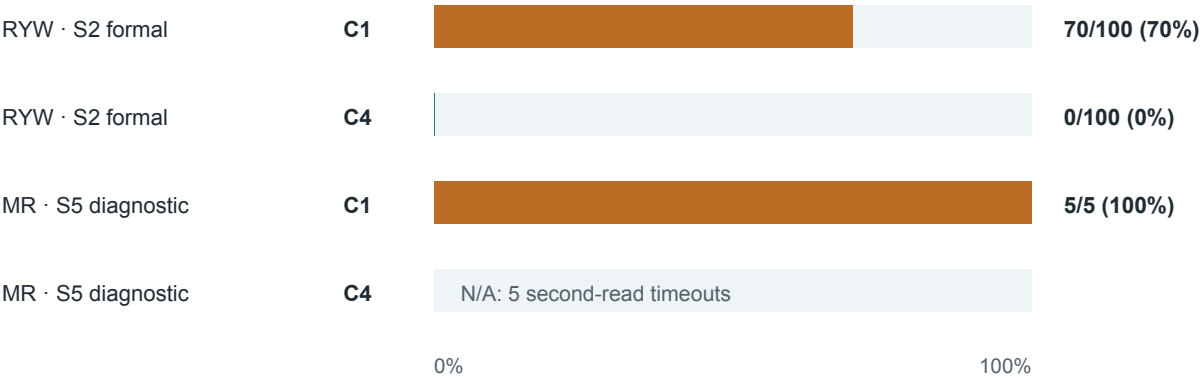
5.2 Secondary loss and loss of majority

Table 6. Formal S2/S3 results

Scenario/model	C1 violations / evaluable	C4 violations / evaluable	Availability and recovery
S2 RYW	70/100	0/100	200/200 trials completed workload and recovery
S2 MW	0/100	0/100	All planned writes succeeded; 200/200 recovered
S2 MR	0/100	0/100	400 workload reads succeeded; 200/200 recovered
S2 WFR	0/100	0/100	400/400 workload operations; 200/200 recovered
S3 immediate MW	0/99	N/A (0/100)	C1: one timeout; C4: 100 first-write timeouts
S3 immediate RYW	0/100	N/A (0/100)	C4: 99 timeouts and one failure
S3 settled MW/RYW	N/A	N/A	Each model/config group: 100 inconclusive attempts

The 1,200 Secondary-related formal attempts comprise S2 RYW 200 + S2 MW 200 + S3 immediate RYW 200 + S3 immediate MW 200 + S3 settled RYW 200 + S3 settled MW 200 [9,11]. S2 RYW is the clearest controlled contrast: C1 returned stale successful reads, while C4 returned none. In S3, zero evaluable C4 trials are availability results rather than zero-percent violation rates.

Stale reads and regressions by evidence cohort



Bars show violations / evaluable trials. Diagnostic and formal samples are separate.
Figure 2. Key stale-read and regression outcomes. Formal and diagnostic cohorts remain separate; MR C4 has no evaluable denominator.

5.3 Primary crash and partition evidence

The formal MR S4 batches completed 100 C1 and 100 C4 trials. Both produced 0/100 regressions, no timeout or error, and 100/100 recovery. Successful read p95, pooling first and second successful reads within each group, was 1.186 ms for C1 and 0.914 ms for C4. These values do not measure election time because the second read was issued only after the topology gate [10].

The targeted MR S5 diagnostic created a verified version difference. C1 produced a 2→1 regression in all five trials. C4 produced no evaluable pair because all five second reads timed out. This supports the bounded prediction that weak settings can return older state, while the causal configuration can wait or become unavailable rather than returning a successful stale result. Five trials are diagnostic evidence, not a population estimate.

Historical MW S4 batches used 20 attempts per configuration and found no confirmed order violation. S5 diagnostics separated rollback from order: the C1 cohort contained rollback outcomes, while the C4 cohort lacked an evaluable consistency denominator. RYW S4/S5 evidence contains only one pilot attempt per configuration and is therefore trace-level evidence.

5.4 Fixed WFR fault matrix

Table 7. WFR v2 formal results

Scenario	C1 violations / evaluable	C4 violations / evaluable	Workload operation success	Recovery
S2 Secondary stop	0/100	0/100	400/400	200/200
S4 Primary crash	0/100	0/98	398/400; two C4 write timeouts	200/200
S5 replication partition	0/98	0/100	400/400; two C1 audit-inconclusive	200/200

Across 600 fixed formal attempts, 596 were evaluable and no confirmed or candidate WFR violation was observed. Four attempts were inconclusive, 1,198/1,200 workload operations succeeded, and 600/600 trials recovered. The separate 120-attempt pilot produced 117 evaluable trials, no violation, three inconclusive outcomes, 239/240 successful operations, and 120/120 recoveries. No failed or inconclusive attempt was replaced [12].

WFR v2 formal classifications

100 attempts per group; 596 evaluable and 4 inconclusive overall

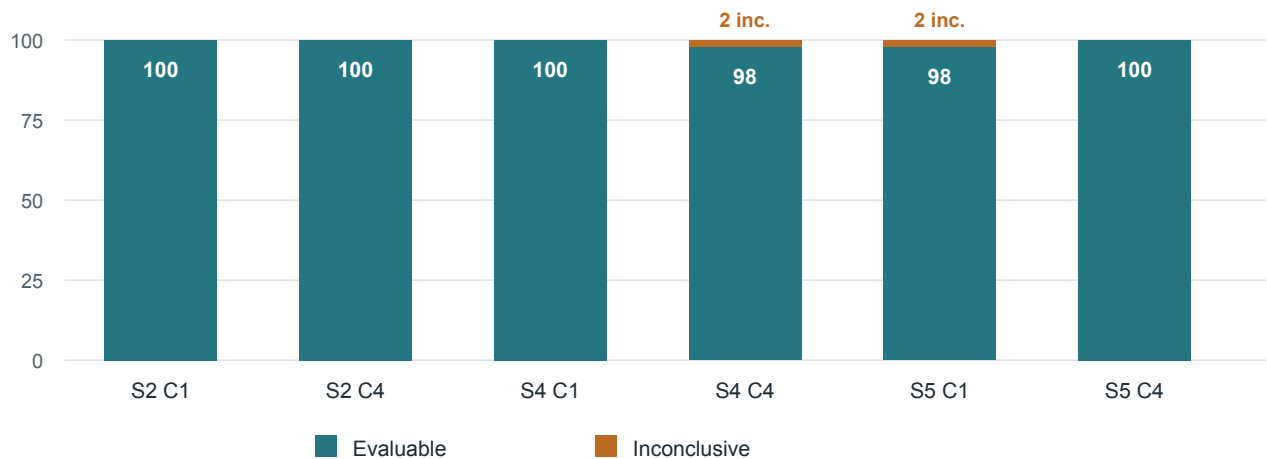


Figure 3. WFR v2 formal classifications. The denominator excludes four inconclusive attempts and does not convert them into passes.

Successful dependent-write p95 was 2.47/4.60 ms for S2 C1/C4, 10,193.41/10,198.75 ms for S4, and 2.44/1,958.36 ms for S5. S4 includes driver waiting across election, whereas S5 waits for the majority-side replacement Primary outside the measured write. Cross-scenario latency differences therefore describe protocol timing and are not an isolated benchmark of concern settings.

6. Comparison with predictions

Table 8. Prediction-observation comparison

Prediction	Observation	Assessment
C1 can expose stale Secondary reads	RYW S2: 70/100 stale; MR S5: 5/5 regressions	Supported for the controlled workloads
C4 should not return successful causally older state when prerequisites hold	No C4 stale RYW in S2; MR S5 timed out; no C4 WFR violation in 298 evaluable formal trials	Consistent; finite evidence is not proof
Majority loss should reduce availability	S3 C4 MW/RYW produced zero evaluable formal trials	Supported; N/A is not a consistency pass
Sequential modifying writes should remain monotonic	No confirmed MW order violation in evaluated normal/S2/failover cohorts	Supported within tested protocols
Majority durability and causal ordering differ	C3 had one stale RYW observation in legacy normal data	Consistent; legacy schema limits attribution
Recovery does not imply consistency	Rollback and timeout occurred although cluster health recovered	Supported; recovery is reported separately

Configuration cannot be interpreted independently of the operation protocol. C1 can appear strong while all nodes are synchronized, yet a targeted lag or partition reveals a permitted regression. C4 can yield no evaluable result when the required majority or causal dependency is unavailable. Counting timeout as consistency success would reverse the experiment's meaning.

7. Limitations and threats to validity

- **Single-host failure domain:** all database containers share one physical host, kernel, storage subsystem, and Docker VM. Container faults do not reproduce independent disks, clocks, or switches.
- **Finite and uneven sampling:** several historical S4/S5 cohorts contain only one, five, or twenty attempts. No power analysis supports population-level probability claims.
- **Protocol-specific fault windows:** MR injects faults between reads, whereas WFR v2 first commits and converges its dependency. Results do not cover every possible partition schedule.
- **Committed-history WFR scope:** all-node seed convergence lowers the chance of missing-history observations and cannot establish a general C1 causal guarantee.
- **Legacy data quality:** early RYW logs lack reliable operation-level latency and are excluded from latency claims.
- **Routing controls:** direct or tagged Secondary access was used only in identified diagnostics to expose version differences.
- **Successful-latency selection:** p95 excludes failed and timed-out operations, so success and timeout counts accompany latency.
- **Clock boundaries:** host and runner wall clocks are not assumed synchronized; matched receipts and monotonic timestamps bound program order, not the physical failure instant.
- **Sequential configuration order:** some fixed batches ran C1 before C4, so cache, thermal, and background-load effects were not randomized.

8. Reproduction package

Repository: https://github.com/ZYKuni/NUS_DSA5208_project1. This revision is anchored to origin/main commit 634b9747130d6a3fc6fe71ca29a5c17596c0658b and unified snapshot results/summary/unified-0922-wfr-final/. Per-run manifests retain the exact experiment source hash, so the report commit is an integration anchor rather than a replacement for run provenance.

A clean reproducer should build the pinned runner, start the normal cluster, and verify one Primary and two Secondaries. Fault experiments run serially against the isolated fault project and use a new experiment ID. The following commands reproduce software validation and the read-only summary without injecting a fault:

```
docker build -t dsa5208-experiment-runner:local \
  -f docker/runner.Dockerfile .
./scripts/start-cluster.sh
./scripts/cluster-status.sh
python -m unittest discover -s tests -v
for d in results/raw/c-wfr-v2-*--0922; do
  python scripts/validate-wfr-run.py "$d"
done
python -m analysis.unified --output results/summary/reproduced-0922
python -m analysis.plot_results \
  --input results/summary/reproduced-0922 \
  --output results/figures/reproduced-0922
```

The 22 September source state passed 117 unit tests, including oracle edge cases, dispatch compatibility, archived-run validation, and tamper rejection [11,12]. Unit tests verify software behavior and are not counted as database observations. An actual fault run should follow report/REPRODUCE-REVIEW.md and the model-specific runner documentation; recovery failure stops the batch, and interrupted output is retained.

9. Team contributions and AI use

Zhao Yikun led deployment, replica-set and fault-lab setup, common fault orchestration, MW experiments, Secondary-loss experiments, evidence validation, and deployment documentation. Zhou Jiahao led RYW and MR design and execution, completed MR S2/S4 formal sampling, documented model-specific interpretation, and independently reproduced a small S2 MW workflow. Chen Zheke led WFR protocol design and execution, conservative WFR classification, unified analysis and figures, and report integration. All members are responsible for reviewing the final claims and submitted artefacts.

An OpenAI coding assistant supported code review, implementation suggestions, synthetic tests, command execution, evidence checking, figure generation, and drafting. Team members selected the questions, operated the database experiments, reviewed the observations, and remain responsible for the interpretation. AI-generated prose and synthetic fixtures were not treated as empirical evidence; quantitative claims trace to archived database logs and validation output.

10. Conclusion

MongoDB's tunable controls produced an observable trade-off between stale success and unavailable operations. Weak local, $w=1$, Secondary-preferring access exposed stale RYW and MR results under controlled lag or partition. Majority concerns plus an explicit causal session produced no successful stale observation in the corresponding tested cohorts, but waited or timed out when the required history or majority was unavailable. Sequential modifying writes remained monotonic in the evaluated MW trials, while rollback remained a separate durability event. The committed-history WFR matrix observed no violation in 596 evaluable formal trials and four inconclusive attempts.

The main methodological conclusion is that consistency, durability, operation availability, and cluster recovery need separate denominators. A recovered cluster does not retroactively make a timed-out operation successful, and a zero-evaluable group cannot support a zero-percent violation claim. Within these boundaries, the observations agree with the pre-registered predictions and demonstrate how concerns, routing, sessions, and fault timing jointly shape client-visible consistency.

References

- [1] MongoDB, Inc. “Causal Consistency and Read and Write Concerns.” MongoDB Manual. <https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/> (accessed 23 September 2026).
- [2] MongoDB, Inc. “Read Isolation, Consistency, and Recency.” MongoDB Manual. <https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/> (accessed 23 September 2026).
- [3] MongoDB, Inc. “Read Concern majority.” MongoDB Manual. <https://www.mongodb.com/docs/manual/reference/read-concern-majority/> (accessed 23 September 2026).
- [4] MongoDB, Inc. “Read Preference.” MongoDB Manual. <https://www.mongodb.com/docs/manual/core/read-preference/> (accessed 23 September 2026).
- [5] MongoDB, Inc. “Write Concern.” MongoDB Manual. <https://www.mongodb.com/docs/manual/reference/write-concern/> (accessed 23 September 2026).
- [6] MongoDB, Inc. “Replication.” MongoDB Manual. <https://www.mongodb.com/docs/manual/replication/> (accessed 23 September 2026).
- [7] MongoDB, Inc. “ClientSession.” PyMongo API Documentation. https://pymongo.readthedocs.io/en/stable/api/pymongo/client_session.html (accessed 23 September 2026).
- [8] Project team. Deployment and reproduction procedure. docs/Aodex/deployment-and-reproduction.md, commit 634b9747.
- [9] Zhao Yikun. Completed Secondary-fault matrix and evidence notes. docs/Aodex/secondary-results-20260915.md, repository archive.
- [10] Zhou Jiahao. MR node-failure formal results. docs/Bodex/mr-node-failure-results-20260918.md, repository archive.
- [11] Project team. Unified evidence summary and provenance catalog. results/summary/unified-0922-wfr-final/, commit 634b9747.
- [12] Chen Zheke. WFR fixed-plan method, results, and validation record. docs/Codex/wfr-results-0922.md and report/wfr-sampling-plan-0922.md.